Entwicklung eines Programmier-Lernspiels

Konzeption, Implementierung und Evaluation des Lerneffektes in einer gamifizierten Umgebung

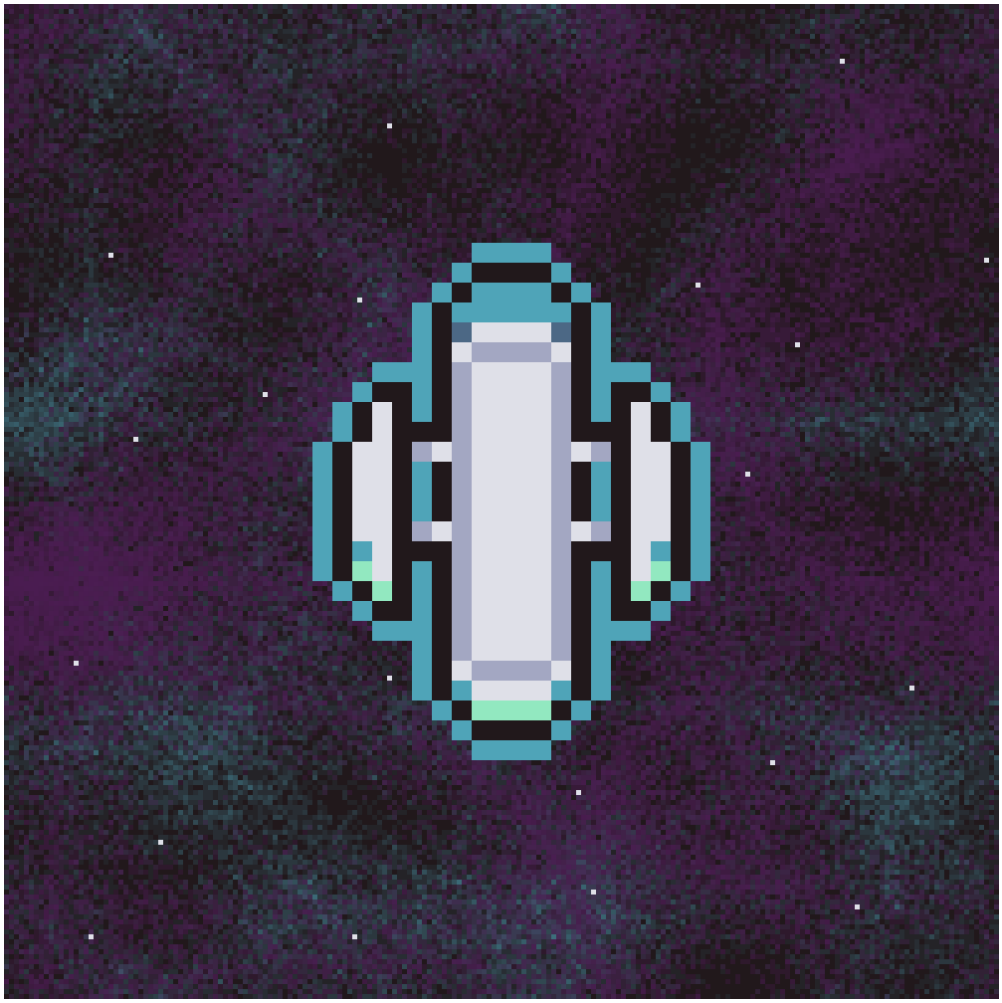

Abbildung 1 – Spiel-Icon, Mattis Auge

Mattis Auge

Schuljahr-2025/26

Grundkurs: Informatik

Manuel Meinhardt

Freiherr-vom-Stein-Gymnasium Hamm

Inhaltsverzeichnis

1. Einleitung....1
1.1 Relevanz und Zielsetzung....1
1.2 Verwandte Spiele und Inspiration....1
2. Theorethische Grundlagen....2
2.1 Gamifizierung und Lernspiele....2
2.2 Motivation durch Wettbewerb....3
3. Konzeption....4
3.1 Spielidee und Designentscheidungen....4
3.2 Spielmechaniken....5
4. Implementierung....5
4.1 Technische Grundlagen....5
4.2 Spiellogik und Steuerung....6
4.3 Umsetzung der Spielerschnittstelle....7
5. Evaluation des Lerneffekts....8
5.1 Methodik....8
5.2 Ergebnisse....9
5.3 Kritische Reflexion....11
6. Fazit und Ausblick....12
Literaturverzeichnis....13
Eidesstaatliche Erklärung....i
Anhang....iii

1. Einleitung

1.1 Relevanz und Zielsetzung

In der heutigen Zeit sind Computerspiele ein Bestandteil des Lebens vieler Teenager. Dabei dienen sie meist nur zur Unterhaltung, doch sie haben auch anderweitig Potenzial. Unterricht wird heutzutage oft „gamifiziert“, also spielerisch gestaltet, und einige Computerspiele existieren nur fürs Lernen[1]. Diese sogenannten Lernspiele werden teilweise bereits in Grundschulen eingesetzt.

Ein besonders vielversprechender Anwendungsbereich ist das Erlernen von Programmierkenntnissen. Da die Informatik in unserer zunehmend digitalisierten Welt an Bedeutung gewinnt, stellt sich die Frage, wie der Einstieg in das Programmieren möglichst zugänglich und motivierend gestaltet werden kann. Sogenannte ‚Coding Games‘ verbinden dabei spielerische Elemente mit dem Erwerb grundlegender Programmierfähigkeiten und sprechen insbesondere jüngere Zielgruppen an.

1.2 Verwandte Spiele und Inspiration

Ein Beispiel für ein erfolgreiches Coding Game ist CodeCombat. Es vermittelt dem Spieler schrittweise grundlegende Programmierkenntnisse. Dabei steuert der Spieler einen Charakter durch einen Kerker mit verschiedenen Monstern, wobei sämtliche Bewegungen des Charakters im Voraus programmiert werden müssen[2]. Das Spiel überzeugt durch ein motivierendes Konzept und bietet dem Spieler einen hohen Lerneffekt. Neben CodeCombat existieren zahlreiche weitere Coding Games mit ähnlichen Ansätzen. Ein Aspekt, der jedoch in vielen populären Beispielen fehlt, ist die Kompetitivität. Kompetitivität steigert Motivation, wenn Wettbewerb als Herausforderung erlebt wird und das Kompetenzgefühl stärkt[3]. Zusätzlich sind nur sehr wenige Coding Games darauf ausgerichtet, auch für erfahrene Programmierer

1 Deterding, Sebastian / Dixon, Dan / Khaled, Rilla / Nacke, Lennart: From Game Design Elements to Gamefulness. Defining “Gamification” (29.09.2011), in: Proceedings of the 15th International Academic MindTrek Conference, https://doi.org/10.1145/2181037.2181040 (aufgerufen am 22.02.2026), S. 9–15.

2 CodeCombat Inc.: CodeCombat – Learn Python, JavaScript, and HTML by Playing a Game (o. D.), https://codecombat.com (aufgerufen am 22.02.2026), S. 1–2.

3 Deci, Edward L. / Ryan, Richard M.: The “What” and “Why” of Goal Pursuits. Human Needs and the Self-Determination of Behavior (2000), in: Psychological Inquiry 11(4), S. 227–268.

interessant zu sein. Das Ziel dieser Facharbeit ist, ein Programmierspiel, das sowohl für Neuanfänger als auch für erfahrene Entwickler interessant ist und Kompetetivität einbaut, zu entwickeln.

2. Theorethische Grundlagen

2.1 Gamifizierung und Lernspiele

Gamifizierung bedeutet, spielerische Konzepte auf spielfremde Kontexte zu übertragen. Dabei werden typische Spielelemente wie Punkte, Bestenlisten, Abzeichen oder Fortschrittsbalken in nicht-spielerische Umgebungen integriert. Ein Beispiel wäre eine Mathematik-Lernplattform, bei der Schüler für gelöste Aufgaben Punkte sammeln und in Ranglisten aufsteigen. Das Ziel dabei ist, die Motivation zu steigern, indem das Lernen durch diese spielerischen Anreize ansprechender gestaltet wird.

Lernspiele, die auch als Serious Games bezeichnet werden, sind hingegen vollständige Spiele, die hauptsächlich auf das Lernen ausgerichtet sind. Im Gegensatz zur Gamifizierung, bei der nur einzelne Spielelemente hinzugefügt werden, ist hier das Spielen selbst die Hauptaktivität, während das Lernen nahtlos in die Spielmechanik integriert ist[4]. Ein bekanntes Beispiel ist die Sprachenlern-App Duolingo, die Sprachen in kurzen, spielerischen Lektionen vermittelt. Nutzer machen Fortschritte durch ihren Kurs, steigen in Ranglisten auf und versuchen, Herausforderungen abzuschließen[5].

Die Vorteile beider Ansätze sind offensichtlich. Sie fördern Motivation, indem sie sofortiges Feedback bieten und erlauben risikofreies Ausprobieren. Studien belegen, dass gut gestaltete Lernspiele die extrinsische Motivation erhöhen können. Die Lernelemente müssen allerdings gut integriert und möglichst unauffällig sein, da es sonst wieder abschreckt.[6]

4 Prensky, Marc: Digital Game-Based Learning (2001), New York: McGraw-Hill, S. 1–20.

5 Jiang, Xiangying / Zalles, Dan / Vesselinov, Roumen: Duolingo Effectiveness Study. Final Report (2012), https://theowlapp.health/wp-content/uploads/2022/04/DuolingoReport_Final-1.pdf (aufgerufen am 22.02.2026), S. 1–10.

6 Ninaus, Manuel / Greipl, Sophie / Kiili, Kristian / Moeller, Korbinian: Game-based Learning und Motivation – Chancen und Grenzen digitaler Lernspiele. In: Schneider, Jan / Niegemann, Helmut (Hrsg.): Handbuch Digitale Bildung. Wiesbaden: Springer VS, 2020, S. 215–232.

2.2 Motivation durch Wettbewerb

Es gibt zwei wichtige Arten von Motivation: intrinsische und extrinsische. Intrinsische Motivation beschreibt Handlung um ihrer selbst willen. Es wird also aus Spaß, Interesse oder Neugier gehandelt. Extrinsische Motivation beschreibt Handlung wegen äußerer Anreize, wie zum Beispiel Belohnungen oder Anerkennung. Lernspiele sollten im besten Fall die intrinsische Motivation fördern. Es gibt drei Grundbedürfnisse, die diese fördern. Die Kompetenz, die Autonomie und die Soziale Eingebundenheit[7]. Wettbewerb ist in der Lage, jedes dieser Bedürfnisse anzusprechen. Wettbewerb erfüllt im Lernkontext eine wichtige motivationale Funktion, indem er Lernende dazu anregt, sich mit anderen zu messen und dadurch ihre eigene Leistung zu hinterfragen. Diese Herausforderung durch den Vergleich mit anderen wirkt als natürlicher Antrieb, da sie den Ehrgeiz weckt und das Bedürfnis anspricht, sich zu verbessern. Gleichzeitig steigert Wettbewerb auch das Engagement und die Anstrengungsbereitschaft der Lernenden, da ein klares Ziel vor Augen steht. Durch messbare Leistungsverbesserungen, die im Wettbewerb sichtbar werden, entsteht zudem ein starkes Kompetenzerleben: Lernende erfahren direkt, dass ihre Bemühungen Früchte tragen. Darüber hinaus besitzt der Wettbewerb eine wichtige soziale Komponente, denn er setzt Interaktion mit anderen Spielern oder Lernenden voraus und fördert so das Gefühl der sozialen Eingebundenheit[8].

Der Psychologe Mihaly Csikszentmihalyi beschreibt mit dem Begriff „Flow" einen Zustand völligen Aufgehens in einer Tätigkeit, in dem Konzentration und Handeln mühelos ineinandergreifen. Dieser Zustand entsteht dann, wenn die Anforderungen einer Aufgabe in einem ausgewogenen Verhältnis zu den Fähigkeiten der Person stehen. Ist eine Aufgabe zu leicht, führt dies zu Langeweile und sinkendem Interesse; ist sie hingegen zu schwer, entsteht Frustration und die Motivation bricht ein[9]. Wettbewerb kann den sogenannten Flow-Zustand gezielt fördern, indem er durch adaptive Schwierigkeit genau jene Balance herstellt, die für das Erleben von Flow notwendig ist.

---

7 Ryan, Richard M. / Deci, Edward L.: Intrinsic and Extrinsic Motivations. Classic Definitions and New Directions (2000), in: Contemporary Educational Psychology 25(1), S. 54–67.

8 Eisenkopf, Gerald: Wettbewerb und Motivation im Unterricht (2010), in: Wirtschaft und Erziehung 62(3), S. 81–86.

9 Csikszentmihalyi, Mihaly: Finding Flow. The Psychology of Engagement with Everyday Life (1997), New York: Basic Books, S. 29–45.

Wettbewerb geht auch mit einer Reihe positiver Effekte auf das Lernen einher. Er steigert Aufmerksamkeit und Konzentration, da das Wettkampfgeschehen erhöhte Präsenz erfordert. Zugleich entsteht eine starke Motivation zur Verbesserung der eigenen Fähigkeiten, weil der Vergleich mit anderen deutlich macht, wo noch Potenzial liegt. Langfristig kann Wettbewerb außerdem zu einer dauerhaften Bindung an die Lernumgebung führen, da der Wunsch, sich weiterzuentwickeln und besser abzuschneiden, immer wieder zur Rückkehr animiert[10].

Trotz dieser Vorteile birgt Wettbewerb auch erhebliche Risiken mit sich. Wiederholte Niederlagen können zu Demotivation führen, insbesondere wenn Lernende das Gefühl entwickeln, grundsätzlich unterlegen zu sein. In solchen Fällen verlagert sich der Fokus häufig vom Lernziel hin zu einem reinen Gewinnen-Wollen, was den pädagogischen Mehrwert untergräbt. Besonders schwächere Lernende sind gefährdet, da sie strukturell benachteiligt werden und schnell den Anschluss verlieren können. Es ist daher wichtig, eine Balance zwischen kooperativen und kompetitiven Elementen herzustellen, damit Wettbewerb inklusiv und motivierend wirkt, ohne einzelne Lernende auszugrenzen oder zu überfordern[11].

3. Konzeption

3.1 Spielidee und Designentscheidungen

Das geplante Spielkonzept ist ein Programmierspiel, in dem zwei oder mehr Spieler gegeneinander antreten. Eine erste Idee war ein Wettrennen, indem zwei Spieler versuchen, als erster ins Ziel zu kommen Dieses Konzept bietet jedoch nicht genügend Varianz, um die Spieler zum Schreiben fortgeschrittener Skripte zu motivieren. Zwar könnte durch verschiedene Karten mehr Abwechslung geschaffen werden, dies würde jedoch einen zu hohen Zeitaufwand bedeuten und den Rahmen dieser Facharbeit überschreiten.

Eine alternative und hier gewählte Idee ist ein Kampf zwischen zwei Spielfiguren. In diesem Szenario müssten die Spieler ihre Strategien an das Verhalten des Gegners anpassen, wodurch eine interessante Simulation entsteht. Da das Spiel auch dem Lernen

10 Reeve, Johnmarshall: Understanding Motivation and Emotion. 7th Edition (2018), Hoboken: Wiley, S. 155–190.

11 Ames, Carole: Classrooms. Goals, Structures, and Student Motivation (1992), in: Journal of Educational Psychology 84(3), S. 261–271.

dienen soll, muss auf ein ‚realistisches' Kampf-Szenario verzichtet werden. Stattdessen fiel die Wahl auf ein Weltraumthema, bei dem die Spielfiguren Raumschiffe sind, die sich gegenseitig mit Lasern angreifen.

3.2 Spielmechaniken

Das erste Problem war hier die Ausrüstung des Raumschiffes. Dieses sollte mehrere Triebwerke und Waffen haben. Ein Triebwerk links und rechts, welche zur Drehung dienen und ein starkes am Heck. Das Schiff sollte einen ausrichtbaren Laser oben haben, und einen Speer oder ähnliches vorne. Dadurch entsteht weitere Varianz, da verschiedene Strategien möglich werden. Jedes Stück Ausrüstung soll hierbei Energie verbrauchen, welche langsam regeneriert. Damit wird vorsichtige Planung belohnt und aggressive Strategien bestraft. Anders als bei zum Beispiel CodeCombat, welches Spieler ihre Bewegungen im Voraus hardcoden lässt, müssen Spieler hier dynamische Skripte, die jeden Frame aufgerufen werden, programmieren. Die Programmierschnittstelle sollte einfach nutzbar sein, weswegen ein In-Game-Editor zur Verfügung stehen sollte.

4. Implementierung

4.1 Technische Grundlagen

Das Spiel wird mit der Godot Game Engine in der Version 4.6 (stable) entwickelt. Ein großer Vorteil dieser Engine ist, dass sie kostenlos und Open-Source ist[12]. Sie unterstützt sowohl zweidimensionale als auch dreidimensionale Spieleentwicklung und nutzt die integrierte Programmiersprache GDScript. GDScript hat eine ähnliche Syntax wie Python und ist damit vergleichsweise einfach und selbst für Anfänger gut verständlich. Alternativ besteht durch Plugins auch die Möglichkeit, in vielen verschiedenen Sprachen zu programmieren.

Godot bietet eine umfangreiche Dokumentation und eine aktive Community, die unter anderem ein hilfreiches Forum betreibt. Der integrierte Editor vereinfacht den Entwicklungsprozess erheblich, und Funktionen wie die eingebaute Physik-Engine oder das Szenen-System nehmen viele aufwendige Aufgaben ab[13].

12 Godot Engine: Documentation – About, https://bit.ly/46kA0HN (aufgerufen am 22.02.2026)

13 Godot Engine: List of features (o. D.), https://bit.ly/4b4MWEg (aufgerufen am 22.02.2026)

Godot arbeitet mit sogenannten Nodes als Grundbausteine für Spielumgebungen. Diese Knoten lassen sich hierarchisch anordnen und miteinander kombinieren. Scenes sind Sammlungen von Nodes, die wiederverwendbar sind und selbst wieder als Knoten instanziiert werden können. Alle Nodes sind dabei mit dem Root-Node, dem sogenannten Tree, verbunden, auf den jedes Skript zugreifen kann[14].

Für dieses Projekt sind folgende Knotentypen von zentraler Bedeutung: Der CharacterBody2D dient als Basis für bewegliche Spielobjekte mit integrierter Physik. Dieser benötigt eine CollisionShape2D, die den Kollisionsbereich definiert. Die visuelle Darstellung des Raumschiffs erfolgt durch ein Sprite2D, das die entsprechende Grafik oder Textur anzeigt. Eine Area2D wird verwendet, um Überlappungen zu erkennen, beispielsweise für Laser oder Schilde[15]. Zusätzlich kommen Timer für zeitgesteuerte Ereignisse zum Einsatz.

An den CharacterBody2D wird außerdem ein Skript angehängt, das die Steuerungslogik implementiert. Weitere Module wie Waffen oder Triebwerke können als separate Child-Nodes ergänzt werden. Diese modulare Struktur ermöglicht eine klare Trennung von Darstellung, Physik und Logik, macht Komponenten wiederverwendbar und erlaubt eine einfache Erweiterbarkeit des Spiels.

4.2 Spiellogik und Steuerung

Jede Fähigkeit des Schiffs wird in einer separaten Funktion geregelt. Die Ship-Szene besteht dabei aus dem CharacterBody2D als Basis, dem Sprite2D-Node für die visuelle Darstellung, drei Partikelsysteme für das linke, das rechte und das mittlere Triebwerk (Left Thruster, Right Thruster, Middle Thruster), einer Laser-Node und einem Knoten namens CodeObject, welcher sich um die Ausführung des Spieler-Codes kümmert. Jede dieser Komponenten besitzt zudem eine CollisionShape2D für die Kollisionserkennung.

Die ready-Funktion ist eine integrierte Godot-Funktion, welche ausgeführt wird, sobald das Schiff im Tree initialisiert ist[16]. Darin wird hier die Spieler-ID gesetzt, die bestimmt, welches Spielerschiff es ist. Mit dieser ID wird die Position des Schiffs festgelegt. Spieler

14 Godot Engine: Nodes and scenes (o. D.), https://bit.ly/nodes-scenes (aufgerufen am 22.02.2026)

15 GeneralistProgrammer: Godot Common Nodes Cheat Sheet https://bit.ly/godot-nodes (aufgerufen am 22.02.2026)

16 Godot Engine: Node (o. D.), https://tinyurl.com/mrxtnn8e (aufgerufen am 22.02.2026)

0 spawnt im oberen Bereich mit zufälligen Koordinaten, während Spieler 1 im unteren Bereich erscheint.

Die Bewegung des Schiffs wird in der physics_process-Funktion verwaltet, die jeden Frame aufgerufen wird. Hier wird zunächst der Code des Spielers ausgeführt, indem eine Funktion des CodeObjects aufgerufen wird. In einem realistischen Weltraumszenario würden die Schiffe keine Reibung haben, und sich nur selber ausbremsen können. Da das Spiel aber nicht zu schwer sein sollte, wird Geschwindigkeit mit linearer Interpolation (lerp) jeden Frame verringert damit das Schiff langsam automatisch bremst.

Die drei Triebwerke haben unterschiedliche Funktionen und werden durch die Funktionen midthruster, leftthruster und rightthruster gesteuert. Das mittlere Triebwerk löst einen starken Schub in Flugrichtung aus, mit einer Kraft von 5 mal dem angegebenen Kraft-Wert. Die seitlichen Triebwerke dienen hauptsächlich der Drehung. Das linke Triebwerk erhöht die Rotationsgeschwindigkeit um 5 mal Power, während das rechte sie entsprechend verringert. Sie erzeugen auch einen kleinen Vorwärtsschub. Die meiste Geschwindigkeit gewinnt man also, wenn man alle Triebwerke gleichzeitig aktiviert. Alle Triebwerke akzeptieren nur Power-Werte zwischen 0 und 1.

Das Waffensystem besteht aus einem drehbaren Laser. Die turnlaser-Funktion erlaubt es, den Laser um 1 Grad pro Aufruf nach links oder rechts zu drehen, abhängig vom übergebenen Parameter. Die shoot-Funktion instanziert ein Bullet-Objekt, welches in Laserrichtung schießt. Die Geschwindigkeit des Geschosses setzt sich zusammen aus einer Grundgeschwindigkeit in Schussrichtung plus der doppelten aktuellen Schiffsgeschwindigkeit, sodass Geschosse von einem Schiff, das in die gleiche Richtung schießt, in die es fliegt, schneller sind. Auf das Schießen gibt es einen Cooldown von 0,3 Sekunden. Jede Aktion verbraucht Energie. Das mittlere Triebwerk kostet 10 Energiepunkte, die seitlichen Triebwerke jeweils 1 Punkt, und ein Laserschuss verbraucht 15 Punkte. Wenn nicht genügend Energie vorhanden ist, wird die entsprechende Aktion nicht ausgeführt. Dies zwingt die Spieler dazu, ihre Energie strategisch einzusetzen und nicht alle Systeme gleichzeitig auf Maximum zu betreiben. Energie wird mit einer Rate von 2 pro Sekunde aufgeladen.

Für die Programmierung stehen dem Spieler einige Funktionen zur Datensammlung zur Verfügung. Darunter ist die aktuelle Laser-Drehung und die Energie. Die zwei wichtigsten Hilfsfunktionen sind angle_to_enemy und dist_to_enemy, die jeweils die Richtung zum Gegner und die Distanz zum Gegner angeben.

4.3 Umsetzung der Spielerschnittstelle

Die Umsetzung der Spielerschnittstelle stellt eine der wichtigsten Herausforderungen des Projekts dar, da diese Spielern ermöglichen muss, ihren eigenen Code schreiben zu können, der zur Laufzeit kompiliert und ausgeführt werden muss. Der Code selber wird außerhalb dieses Skripts gespeichert und mit der ID des Spielers, von dem es geschrieben wurde, versehen. Mit der compile_dynamic_code-Funktion wird der Code kompiliert. GDScript erlaubt einem, eine Skriptvariable zu erstellen und in dieser den Source Code einzusetzen. Dem Skript wird dann das UserCode-Objekt weitergegeben, damit es Methoden ausführen kann.

Der Spieler-Code muss eine run-Funktion enthalten, die vom CodeObject aufgerufen wird. Innerhalb dieser Funktion können die Spieler beliebige Logik implementieren. Die einzige Einschränkung ist, dass sie auf die vom CodeObject bereitgestellten Funktionen beschränkt sind und nicht direkt auf die Engine oder andere Nodes zugreifen können. Dies sorgt für Fairness und verhindert, dass Spieler das Spiel auf unvorhergesehene Weise manipulieren können.

5. Evaluation des Lerneffekts

5.1 Methodik

Die Evaluation des Lerneffekts wurde mit einem aufgabenbasierten Ansatz durchgeführt. Dadurch können spezifische Programmierfähigkeiten in einer praxisnahen Umgebung getestet werden.

Als Testpersonen nahmen zwei Schülerinnen teil. Beide haben zwar schon durch Informatikunterricht grundlegende Vorerfahrungen mit Programmierung gehabt, haben diese jedoch nie in größerem Umfang angewendet oder beherrscht. Beiden wurde eine Datei zugeschickt, welche die Methoden des Schiffs aufzählt und beschreibt[17].

Der Test wurde in mehrere Stufen mit steigender Schwierigkeit aufgebaut. Jede Stufe besteht aus konkreten Aufgaben, die bestimmte Spielfunktionen abdecken. Angefangen wurde bei einfachen Grundlagen wie dem Aktivieren einzelner Schiffssysteme. Später wurde Logik mit If-Statements eingeführt, bis dann letztendlich mehrere Systeme gleichzeitig koordiniert werden mussten.

17 Siehe Anhang, Abbildung 2

In Stufe 1 wurden die drei wichtigsten Systeme geprüft. Das Aktivieren des Haupttriebwerks für Vorwärtsbewegung, das Drehen des Schiffs mittels Einzelaktivierung der Seitentriebwerke, und das Abfeuern von Laserschüssen. Diese Aufgaben sollten die Testpersonen mit der Syntax und den verfügbaren Funktionen vertraut machen. Stufe 2 führte bedingte Logik ein. Die zentrale Aufgabe war es, das Schiff automatisch zum Gegner drehen zu lassen, indem die Funktion angle_to_enemy mit if-Bedingungen kombiniert wurde. Stufe 3 enthält Aufgaben wie automatisches Zielen mit dem unabhängig vom Schiff drehbaren Laser, sowie die Kombination mehrerer Systeme gleichzeitig, z.B. das gleichzeitige Drehen zum Gegner, Vorwärtsbewegen und Schießen. Die finale Herausforderung war ein asymmetrisches Szenario. Eine Testperson programmiert ein fliehendes Schiff, während die andere ein verfolgendes Schiff erstellt. Dabei muss technisches Verständnis und strategisches Denken angewendet werden.

Während der gesamten Testphase wurden verschiedene Aspekte dokumentiert. Die Anzahl der benötigten Versuche pro Aufgabe, die Qualität der Lösungsansätze, auftretende Fehler und Missverständnisse, sowie die Bereitschaft und Fähigkeit, auf Hinweise zu reagieren. Der verwendete Code wurde jeweils festgehalten, um die Entwicklung der Programmierlogik nachvollziehen zu können. Zusätzlich wurde notiert, bei welchen Aufgaben die Testpersonen Hilfe benötigten und wo sie eigenständig Lösungen entwickeln konnten.

Dieser Ansatz erlaubt eine qualitative Bewertung des Lerneffekts, indem sie zeigt, welche Programmierkonzepte durch das Spiel erfolgreich vermittelt werden und an welchen Stellen Schwierigkeiten auftreten.

5.2 Ergebnisse

Die Aufgaben der ersten Stufe wurden von beiden Testpersonen problemlos und schnell bewältigt. Sowohl Person 1 als auch Person 2 benötigten jeweils nur einen Versuch, um ihr Schiff vorwärts zu bewegen, es zu drehen und Laserschüsse abzufeuern. Die Lösungen waren dabei korrekt und direkt. Für die Vorwärtsbewegung wurde midthruster mit dem Parameter 1 aufgerufen[18], für die Drehung leftthruster mit demselben Wert[19], und für das Schießen die shoot-Funktion[20]. Dieser Erfolg

18 Siehe Anhang, Abbildung 3

19 Siehe Anhang, Abbildung 4

20 Siehe Anhang, Abbildung 5

demonstrierte, dass beide ein grundsätzliches Verständnis für Syntax haben und von sich aus auf die Dokumentation zugreifen und diese verstehen können. Bei der ersten Aufgabe mit bedingter Logik, also dem automatischen Drehen zum Gegner, zeigten sich erste Unterschiede in den Lösungsansätzen. Person 1 entwickelte einen Code, der zwar funktional war, aber bewusst von der optimalen Lösung abwich. Sie programmierte ihr Schiff so, dass es immer in einem 90° Winkel zum anderen Schiff flog, und es somit umkreiste. Ihr Code enthielt zusätzlich weitere Funktionsaufrufe wie shoot, get_energy und midthruster, was zeigt, dass sie bereits versuchte, mehrere Systeme zu kombinieren, auch wenn dies nicht explizit gefordert war[21]. Person 2 hatte einen konzeptionell richtigen Ansatz, vergaß jedoch einen Teil der Bedingungslogik. Ihr Code prüfte nur, ob der Gegner links oder rechts von ihrem Schiff war. Damit drehte sie sich zwar zum anderen Schiff, hörte allerdings auf sich zu bewegen, sobald sie auf den Gegner schaute[22]. Diese Aufgabe dauerte bei Person 2 etwas länger als die vorherigen, da sie die Logik mehrfach überdenken musste. Die Komplexität stieg bei der Aufgabe des automatischen Zielens deutlich an. Der korrekte Ansatz wäre hier, den aktuellen Winkel zum Gegners mit dem Winkel des Lasers zu subtrahieren. Damit würde sich der Laser in die richtige Richtung drehen. Person 1 baute diese Rechnung zwar ein, änderte sie jedoch basierend auf in welcher Richtung der Gegner war. Wenn der Gegner links von ihr war, addierte sie die beiden Werte stattdessen, was dafür sorgte, dass sich ihr Laser immer weiterdrehte, und selten tatsächlich auf den Gegner zielte[23]. Person 2 machte einen ähnlichen Fehler, da sie die Rechnung nur abfragte, wenn ihr Schiff schon einen Winkel von 0° zum anderen Schiff hatte. Dadurch wurde der Laser nie gedreht[24].

Bei der darauffolgenden Aufgabe, auf den Gegner zuzugehen und gleichzeitig zu schießen, zeigte Person 1 deutlichen Fortschritt. Sie benötigte zwar länger für diese Aufgabe, entwickelte aber eine gut strukturierte Lösung. Ihr Code drehte den Laser basierend auf dem Winkel zum Gegner, aktivierte das entsprechende Seitentriebwerk und feuerte kontinuierlich[25]. Person 2 erarbeitete die Lösung auf eine funktionstüchtige Weise, brauchte dabei aber einige Versuche.

21 Siehe Anhang, Abbildung 6

22 Siehe Anhang, Abbildung 7

23 Siehe Anhang, Abbildung 8

24 Siehe Anhang, Abbildung 9

25 Siehe Anhang, Abbildung 10

Die abschließende asymmetrische Herausforderung offenbarte deutliche Unterschiede zwischen den beiden Testpersonen. Person 1, die das verfolgende Schiff programmieren sollte, erstellte ein funktionsfähiges Skript mit nur minimaler Hilfe. Besonders bemerkenswert ist, dass sie zusätzlich Energieverwaltung einbaute[26]. Dies zeigt ein fortgeschrittenes Verständnis sowohl der Spielmechanik als auch der Programmierlogik. Person 2 hingegen, die das fliehende Schiff programmieren sollte, hatte erhebliche Schwierigkeiten. Ihr Code enthielt zwar gute Ansätze wie die Verwendung von Variablen zur Speicherung von angle_to_enemy und energy, sowie die Verwendung von return um den Code vorzeitig zu beenden und abs für Absolutwerte. Diese Methodiken wurden aber willkürlich und ohne wirklichen Nutzen angewendet. Das gesamte Skript zeigte keine tatsächliche Logik, sondern wirkte so als hätte sie Elemente, an die sie sich ungefähr erinnern konnte, ohne weitere Planung zusammengesetzt[27]. Besonders problematisch war, dass Person 2 auf angebotene Hilfe nicht wirklich einging, was darauf hindeutet, dass sie entweder frustriert war oder die Konzepte nicht ausreichend verstanden hatte um die Hinweise umzusetzen.

5.3 Kritische Reflexion

Die Evaluation des Lerneffekts anhand dieses Versuchs zeigt keine eindeutigen Daten. Die beiden Testpersonen lieferten drastisch unterschiedliche Ergebnisse, was eine allgemeine Aussage über die Wirksamkeit des Spiels deutlich schwieriger macht. Person 1 lernte die Syntax von GDScript grundlegend kennen und war zu einem gewissen Grad in der Lage, zunehmend komplexere Programmieraufgaben eigenständig zu lösen. Person 2 hingegen zeigte keine große Veränderung in ihrem Programmierverständnis. Sie zeigte zwar interessante Ansätze, konnte diese jedoch nicht tatsächlich korrekt zusammensetzen. Der Lerneffekt hing also stark von individuellen Faktoren wie Vorkenntnissen, Lernstil oder Frustrationstoleranz ab.

Das größte Problem war, dass beide Testpersonen keinen Flow-Zustand erreichen konnten. Person 1 zeigte trotz ihres Fortschritts wiederholt Anzeichen von Langeweile und Verwirrung. Für sie waren die Aufgaben also nicht ausreichend herausfordernd oder die Spielmechanik nicht fesselnd genug. Person 2 war sichtlich frustriert, was auf

26 Siehe Anhang, Abbildung 11

27 Siehe Anhang, Abbildung 12

eine Überforderung hindeutet. Kombiniert zeigt sich also, dass das Spiel keine gelungene Balance zwischen Fähigkeiten und Herausforderung herstellen konnte.

Zudem muss die begrenzte Aussagekraft dieser Evaluation betont werden. Mit nur zwei Testpersonen lassen sich keine statistisch validen Schlüsse ziehen. Die Stichprobe ist zu klein, um zu bestimmen, ob die beobachteten Unterschiede auf das Spieldesign, individuelle Unterschiede oder andere Faktoren zurückzuführen sind. Eine größere und diversere Testgruppe wäre notwendig gewesen, um verlässlichere Erkenntnisse zu gewinnen.

Ein weiteres grundlegendes Problem war, dass es beide schwierig fanden, den Kontext zu verstehen, in dem sie programmieren. Die Art, wie der Spieler-Code in das Spielgeschehen integriert wird, war somit nicht intuitiv genug.

6. Fazit und Ausblick

Zusammenfassend lässt sich aussagen, dass ein Programmierspiel nicht automatisch ein guter Weg ist, seine Programmierfähigkeit zu verbessern. Die theoretische Auseinandersetzung sowie die praktische Umsetzung zeigen, dass gamifizierte Elemente wie Wettbewerb und Herausforderungen grundsätzlich lernfördernde Bedingungen schaffen können.

Das geplante Konzept konnte erfolgreich mit der Godot Engine umgesetzt werden. Die Evaluation zeigt jedoch stark unterschiedliche und unvollständige Ergebnisse. Neben den gamifizierten Elementen müssen weitere Faktoren wie die intuitive Verständlichkeit des Kontextes und die Balance zwischen Herausforderung und Überforderung beachtet werden. Da diese Elemente nicht umgesetzt werden konnten, ist das Spiel kein erfolgreiches Lernspiel, und Spieler können keinen Flow-Zustand erreichen.

Zukünftige Untersuchungen sollten größere und diversere Testgruppen einbeziehen und gezielt die Wirkung verschiedener Spielmechaniken und Tutorial-Ansätze vergleichen. Ein Vorher-Nachher-Test würde vermutlich objektivere Vergleichswerte liefern. Ebenso wäre es sinnvoll, das Spiel weiterzuentwickeln und dabei visuelles Feedback oder Einzelspielermodi zu implementieren.

Abschließend lässt sich festhalten, dass das Programmierspiel zwar gute Ansätze hatte, allerdings die identifizierten Designprinzipien zur Erzeugung von Flow zu sehr außen vor lässt. Wenn diese richtig umgesetzt wären, könnte es erfolgreich sein.

Literaturverzeichnis

Primärliteratur (wissenschaftliche Originalstudien und grundlegende theoretische Arbeiten)

Ames, Carole: Classrooms. Goals, Structures, and Student Motivation (1992), in: Journal of Educational Psychology 84(3), S. 261–271.

Csikszentmihalyi, Mihaly: Finding Flow. The Psychology of Engagement with Everyday Life (1997), New York: Basic Books, S. 29–45.

Deci, Edward L. / Ryan, Richard M.: The "What" and "Why" of Goal Pursuits. Human Needs and the Self-Determination of Behavior (2000), in: Psychological Inquiry 11(4), S. 227–268.

Deterding, Sebastian / Dixon, Dan / Khaled, Rilla / Nacke, Lennart: From Game Design Elements to Gamefulness. Defining "Gamification" (29.09.2011), in: Proceedings of the 15th International Academic MindTrek Conference, https://doi.org/10.1145/2181037.2181040 (aufgerufen am 22.02.2026), S. 9–15.

Eisenkopf, Gerald: Wettbewerb und Motivation im Unterricht (2010), in: Wirtschaft und Erziehung 62(3), S. 81–86.

Jiang, Xiangying / Zalles, Dan / Vesselinov, Roumen: Duolingo Effectiveness Study. Final Report (2012), https://theowlapp.health/wp-content/uploads/2022/04/DuolingoReport_Final-1.pdf (aufgerufen am 22.02.2026), S. 1–10.

Prensky, Marc: Digital Game-Based Learning (2001), New York: McGraw-Hill, S. 1–20.

Ryan, Richard M. / Deci, Edward L.: Intrinsic and Extrinsic Motivations. Classic Definitions and New Directions (2000), in: Contemporary Educational Psychology 25(1), S. 54–67.

Sekundärliteratur (Handbücher, zusammenfassende Darstellungen)

Ninaus, Manuel / Greipl, Sophie / Kiili, Kristian / Moeller, Korbinian: Game-based Learning und Motivation – Chancen und Grenzen digitaler Lernspiele. In: Schneider, Jan / Niegemann, Helmut (Hrsg.): Handbuch Digitale Bildung. Wiesbaden: Springer VS, 2020, S. 215–232.

Reeve, Johnmarshall: Understanding Motivation and Emotion. 7th Edition (2018), Hoboken: Wiley, S. 155–190.

Internetquellen und technische Dokumentation

CodeCombat Inc.: CodeCombat – Learn Python, JavaScript, and HTML by Playing a Game (o. D.), https://codecombat.com (aufgerufen am 22.02.2026).

GeneralistProgrammer: Godot Common Nodes Cheat Sheet, https://bit.ly/godot-nodes (aufgerufen am 22.02.2026).

Godot Engine: Documentation – About, https://bit.ly/46kA0HN (aufgerufen am 22.02.2026).

Godot Engine: List of features (o. D.), https://bit.ly/4b4MWEg (aufgerufen am 22.02.2026).

Godot Engine: Node (o. D.), https://tinyurl.com/mrxtnn8e (aufgerufen am 22.02.2026).

Godot Engine: Nodes and scenes (o. D.), https://bit.ly/nodes-scenes (aufgerufen am 22.02.2026).

Eidesstaatliche Erklärung

Eidesstattliche Erklärung

Hiermit versichere ich, ***Vorname Nachname*, geb. am *Tag.Monat.Jahr***, dass ich die vorliegende Arbeit selbstständig, ohne fremde Hilfe und unter Angabe sämtlicher Hilfsmittel verfasst habe.

Ort, Hamm

Tag.Monat.Jahr 22.02.2026

M. Auge

Unterschrift

Freiherr-vom-Stein-Gymnasium Hamm

Jahrgangsstufe Q1

Protokoll zur Facharbeit 2025/2026

Name	Auge	Vorname	Mattis

Fach	LK/GK	Kurslehrer/in
Informatik	GK	Manuel Meinhardt

1. Beratungsgespräch Datum: 18.12.2025

Themen des Gesprächs:	Vereinbarungen:
Rahmenthema der Facharbeit, Nutzung von externen Libraries, Nutzung einer Game Engine	Thema: Entwicklung eines Programmier-Lernspiels. Nutzung von externen Libraries und Game Engines ist erlaubt, es muss nur klar gekennzeichnet sein.
Unterschrift SchülerIn: M. Auge	Unterschrift Lehrkraft:

2. Beratungsgespräch Datum: 16.02.2026

Themen des Gesprächs:	Vereinbarungen:
Einfügung von Code und Bildern des Spiels, Korrekte Zitierweise	Code und Bilder müssen nicht in die Facharbeit eingefügt werden. Jede Aussage, die als Fakt angegeben wird muss belegt sein
Unterschrift SchülerIn: M. Auge	Unterschrift Lehrkraft:

3. Beratungsgespräch Datum:

Themen des Gesprächs:	Vereinbarungen:
Unterschrift SchülerIn:	Unterschrift Lehrkraft:

Anhang

README

Methoden können aufgerufen werden mit e.funktion()

Triebwerke

midthruster(power)

- Aktiviert das Haupttriebwerk am Heck des Schiffs
- `power` : Wert zwischen 0 und 1 (z.B. 0.5 für halbe Kraft)
- Erzeugt starken Schub in Flugrichtung
- Energieverbrauch: 10 pro Aktivierung

leftthruster(power)

- Aktiviert das linke Triebwerk
- `power` : Wert zwischen 0 und 1
- Dreht das Schiff nach links (erhöht Rotation)
- Erzeugt zusätzlich leichten Vorwärtsschub
- Energieverbrauch: 1 pro Aktivierung

rightthruster(power)

- Aktiviert das rechte Triebwerk
- `power` : Wert zwischen 0 und 1
- Dreht das Schiff nach rechts (verringert Rotation)
- Erzeugt zusätzlich leichten Vorwärtsschub
- Energieverbrauch: 1 pro Aktivierung

Waffensystem

turnlaser(dir)

- Dreht den Laser um 1 Grad
- `dir` : negative Werte für links, positive für rechts
- Beispiel: `turnlaser(-1)` dreht nach links, `turnlaser(1)` nach rechts

laser_rotation()

- Gibt die aktuelle Ausrichtung des Lasers zurück
- Rückgabewert: Winkel zwischen 0 und 360 Grad

shoot()

- Feuert ein Geschoss ab
- Cooldown: 0.3 Sekunden zwischen Schüssen
- Energieverbrauch: 15 pro Schuss
- Geschoss fliegt in Richtung der Laser-Ausrichtung

Informationsfunktionen

angle_to_enemy()

- Gibt den Winkel zum gegnerischen Schiff zurück
- Rückgabewert: Winkel zwischen 0 und 360 Grad, relativ zur eigenen Ausrichtung
- 0° bedeutet der Gegner ist direkt vor dir
- 90° bedeutet der Gegner ist rechts von dir
- 180° bedeutet der Gegner ist hinter dir
- 270° bedeutet der Gegner ist links von dir

dist_to_enemy()

- Gibt die Entfernung zum gegnerischen Schiff zurück
- Rückgabewert: Distanz in Pixeln

get_energy()

- Gibt die aktuelle verfügbare Energie zurück
- Maximum: 1000

Abbildung 2 – Informationsdatei, die an die Spieler gesendet wurde

```
func run():
    e.midthruster(1)
```

Abbildung 3 – Lösung der Spieler für Aufgabe 1

```
func run():
    e.leftthruster(1)
```

Abbildung 4 – Lösung der Spieler für Aufgabe 2

```
func run():
    e.shoot()
```

Abbildung 5 – Lösung der Spieler für Aufgabe 3

```
func run():
    if e.angle_to_enemy()<270:
        e.leftthruster(1)
    if e.angle_to_enemy()<90:
        e.rightthruster(1)
    e.shoot()
    e.get_energy()
    e.dist_to_enemy()
    e.midthruster(0.1)
```

Abbildung 6 – Lösung für Aufgabe 3, Person 1

```
func run():
    if e.angle_to_enemy()<0:
        e.leftthruster(1)
    if e.angle_to_enemy()>0:
        e.rightthruster(1)
```

Abbildung 7 – Lösung für Aufgabe 3, Person 2

```
func run():
    if e.angle_to_enemy()<0 && e.angle_to_enemy()>-180:
        e.turnlaser(e.angle_to_enemy() - e.laser_rotation())
    if e.angle_to_enemy()>0&& e.angle_to_enemy()<180:
        e.turnlaser(e.angle_to_enemy() + e.laser_rotation())
    if e.angle_to_enemy() ==0:
        e.shoot()
    e.leftthruster(0.5)
    e.rightthruster(0.5)
    e.get_energy()
    e.dist_to_enemy()
```

Abbildung 8 – Lösung für Aufgabe 4, Person 1

```
var e

func run():
    if e.angle_to_enemy()<0 && e.angle_to_enemy()>-180:
        e.rightthruster(1)
    if e.angle_to_enemy()>0&& e.angle_to_enemy()<180:
        e.leftthruster(1)
    if e.angle_to_enemy()==0:
        e.turnlaser (e.angle_to_enemy()-e.laser_rotation())
    e.shoot()
```

Abbildung 9 – Lösung für Aufgabe 4, Person 2

```
func run():
    if e.angle_to_enemy()<0 && e.angle_to_enemy()>-180:
        e.turnlaser(e.angle_to_enemy() - e.laser_rotation())
        e.leftthruster(1)
        e.shoot()
    if e.angle_to_enemy()>0&& e.angle_to_enemy()<180:
        e.turnlaser(e.angle_to_enemy() - e.laser_rotation())
        e.rightthruster(1)
        e.shoot()
    if e.angle_to_enemy() ==0:
        e.rightthruster(1)
        e.leftthruster(1)
        e.shoot()
```

Abbildung 10 – Lösung der Spieler für Aufgabe 5

```
func run():
    if e.angle_to_enemy()<0 && e.angle_to_enemy()>-180:
        e.turnlaser(e.angle_to_enemy() - e.laser_rotation())
        e.leftthruster(1)
        e.shoot()
    if e.angle_to_enemy()>0&& e.angle_to_enemy()<180:
        e.turnlaser(e.angle_to_enemy() - e.laser_rotation())
        e.rightthruster(1)
        e.shoot()
    if e.angle_to_enemy() ==0:
        if e.get_energy () >=500:
            e.midthruster(1)
            e.shoot()
        else:
            e.rightthruster(1)
            e.leftthruster(1)
            e.shoot()
```

Abbildung 11 – Lösung für Aufgabe 6, Person 1

```
func run():
    var angle = e.angle_to_enemy()
    var Energy = e.get_energy()

    if Energy < 20:
        if angle > 0:
            e.rightthruster(0.5)
        else:
            e.leftthruster(0.5)
        return
    if abs(angle) < 15:
        e.shoot()
    if angle < 0:
        e.rightthruster(1)
    else:
        e.leftthruster(1)
    e.midthruster(1)
```

Abbildung 12 – Lösung für Aufgabe 7, Person 2